



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

WEB TECHNOLOGY (SECJ3483)

LAB ASSIGNMENT: SECURITY INVESTIGATION AND FIX

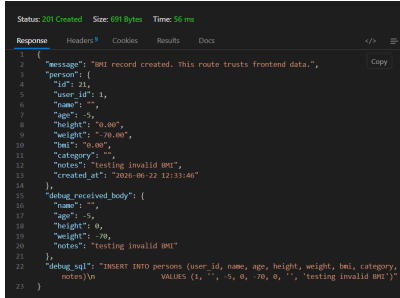
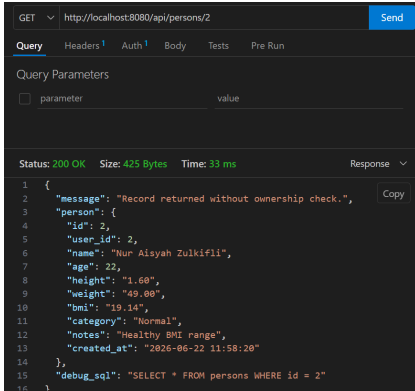
LECTURER :

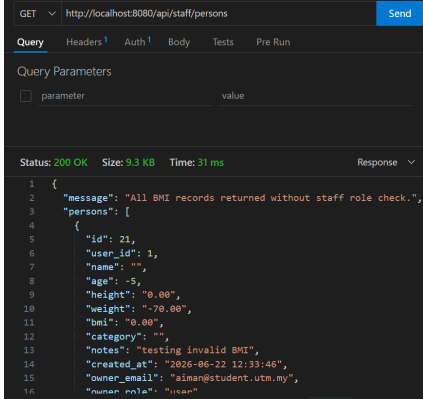
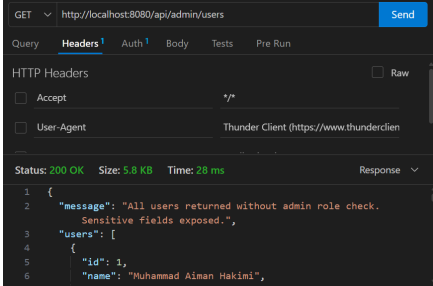
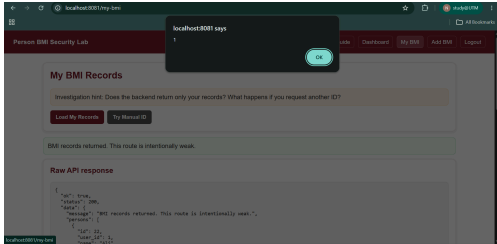
MR. NORIZAM BIN KATMON

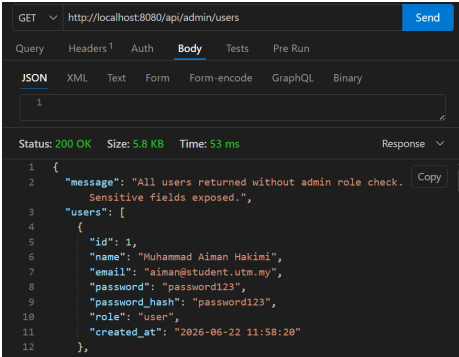
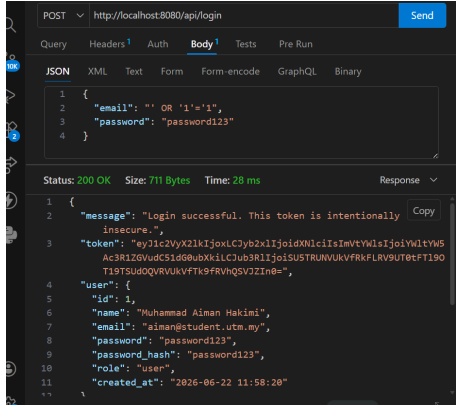
PREPARED BY :

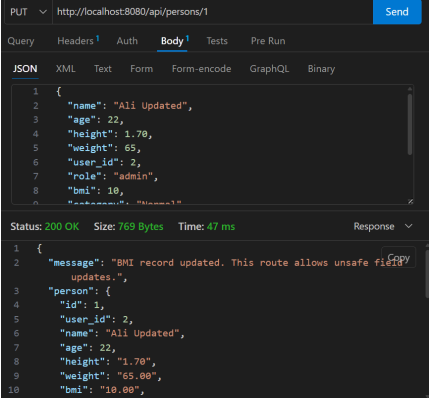
NO.	NAME	MATRIC NUMBER
1.	ARINAH FILZAH BINTI ROSLI	A23CS0050
2.	NATASHA MAISARAH BINTI ZAKI	A23CS0141
3.	LIM XIN ROU	A23CS0240
4.	CHEN WEI JAY NICKOLAS	A23CS5028

Phase 1: Security Investigation

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data	Backend should reject invalid data		Yes	1. The backend accepts the data. The response message confirms successful creation. 2. The frontend validation does not stop the request. The request was successfully sent and processed by the backend. 3. Yes, the same request can be sent using Postman. 4. The validation must be enforced on both frontend and backend. Frontend validation will improve user experience, while backend validation provides security.
2	Access another user's BMI record	Backend should deny access		Yes	1. The backend did return another user's BMI record. 2. The login itself is not enough to deny access as it only verifies the user's identity. 3. The backend should compare the authenticated user's ID that is obtained from the JWT token with the user_id stored in the requested BMI record. 4. The staff and admin should have a different access using the implementation of RBAC.

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
3	Access staff route as normal user	Backend should deny access	 <pre> GET http://localhost:8080/api/staff/persons Status: 200 OK Size: 9.3 KB Time: 31 ms { "message": "All BMI records returned without staff role check.", "persons": [{ "id": 21, "user_id": 1, "name": "", "age": -5, "height": "0.00", "weight": "-70.00", "bmi": "0.00", "category": "", "notes": "testing invalid BMI", "created_at": "2026-06-22 12:33:46", "owner_email": "aiman@student.utm.my", "runner_role": "user" }] } </pre>	Yes	1. The backend did not block the request but returned the complete list of persons and users. 2. Only staff and admin should be allowed to access GET /api/staff/persons. 3. Checking should not be done in the frontend route guard only because it cannot prevent unauthorized access if requests are sent through tools like ThunderClient. 4. The backend side must make the final authorization decision.
4	Access admin route as normal user	Backend should deny access	 <pre> GET http://localhost:8080/api/admin/users Status: 200 OK Size: 5.8 KB Time: 28 ms { "message": "All users returned without admin role check. Sensitive fields exposed.", "users": [{ "id": 1, "name": "Muhammad Aiman Hakini", }] } </pre>	Yes	1. The backend did not block the request but returned the complete list of users. 2. Only admin should be allowed to access GET /api/admin/users. 3. Checking should not be done in the frontend route guard only because it cannot prevent unauthorized access if requests are sent through tools like ThunderClient. 4. The backend side must make the final authorization decision.
5	Submit XSS payload in notes	Payload should display as text	 <pre> { "message": "All BMI records returned without staff role check.", "records": [{ "id": 1, "user_id": 1, "name": "Muhammad Aiman Hakini", "age": -5, "height": "0.00", "weight": "-70.00", "bmi": "0.00", "category": "", "notes": "testing invalid BMI", "created_at": "2026-06-22 12:33:46", "owner_email": "aiman@student.utm.my", "runner_role": "user" }] } </pre>	Yes	1. Yes, the alert is executed. When the BMI record was viewed in the frontend, the browser executed the JavaScript code. 2. The execution of the payload suggests that the frontend is rendering the notes field as HTML using methods such as innerHTML or Vue's v-html. 3. The safer approach is to display user input as plain text using Vue's text interpolation: {{ notes }}.

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
					4. User-generated content should not be rendered as HTML because it may contain malicious scripts that can execute in other users' browsers.
6.	Check API response	Password/hash should not be visible	 <pre> 1 { 2 "message": "All users returned without admin role check. Sensitive fields exposed.", 3 "users": [4 { 5 "id": 1, 6 "name": "Muhammad Aiman Hakimi", 7 "email": "aiman@student.utm.my", 8 "password": "password123", 9 "password_hash": "password123", 10 "role": "user", 11 "created_at": "2026-06-22 11:58:20" 12 } 13] 14 } </pre>	Yes	1. Yes. The API response includes the password_hash field. It also exposes the password field. 2. The frontend does not require password, password_hash, reset_token, or any other sensitive authentication data. These values should remain on the server and never be included in API responses. 3. The API should return only the information necessary for the frontend, such as id, name, email, and role.
7	Try SQL Injection input	Input should be treated as data	 <pre> 1 { 2 "email": "' OR '1'='1", 3 "password": "password123" 4 } </pre> <pre> 1 { 2 "message": "Login successful. This token is intentionally insecure.", 3 "token": "eyJ1c2VyeXZlkiJ0wLCjYb2xliJ0idXNlciIsImVtZWlsIjoIYmItYm5Ac3RlZGVudC5ldG8ubXkiLCJub3RlIjoisUSTRUNUNVkvFRkPLRV9UTeFT190Tl9TSUd0QVRVUkVFTk9FRVhQSVZiIn0=", 4 "user": { 5 "id": 1, 6 "name": "Muhammad Aiman Hakimi", 7 "email": "aiman@student.utm.my", 8 "password": "password123", 9 "password_hash": "password123", 10 "role": "user", 11 "created_at": "2026-06-22 11:58:20" 12 } 13 } </pre>	Yes	1. Yes, the SQL query behaved unexpectedly since the SQL query was altered by the injected input. 2. The login was successful even though the email field contained the SQL injection payload instead of a legitimate email address. 3. The API returned the details of the first matching user including sensitive fields such as password and password_hash. 4. The debug_sql output shows that user input was directly concatenated into the SQL query.

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
8	Try to update protected fields	Backend should reject or ignore protected fields	 <pre> PUT http://localhost:8080/api/persons/1 { "name": "All Updated", "age": 22, "height": 1.70, "weight": 65, "user_id": 2, "role": "Admin", "bmi": 10, "category": "Normal" } Status: 200 OK Size: 769 Bytes Time: 47 ms { "message": "BMI record updated. This route allows unsafe field updates.", "person": { "id": 1, "user_id": 2, "name": "All Updated", "age": 22, "height": "1.70", "weight": "65.00", "bmi": "10.00", } } </pre>	Yes	1. Yes the response confirmed the user_id is updated. 2. No, the generated SQL does not update it because the persons table does not have the role column. 3. The backend did accept bmi and category from the frontend. 4. Only fields that users are permitted to edit should be accepted by the backend, such as name, age, height, weight, and notes.

Phase 2: Weakness Classification and Discussion

Weakness Found

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
XSS payload executes	Frontend	XSS	Browser script execution
SQL Injection input affects query	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or role	API / Backend	Mass assignment	Privilege or ownership manipulation

Discussion Questions

1. Which weakness is the most dangerous? Why?
SQL injection because it allow attackers to read, modify or delete data in the database, bypass authentication and even take control of the database if not properly prevented.
2. Which weakness is caused by trusting the frontend?
Negative weight accepted as the backend assumes the frontend has already validated the input which allows invalid or malicious data to be submitted.
3. Which weakness is caused by missing backend validation?
Negative weight accepted because the server does not verify whether the submitted weight is valid before storing it.
4. Which weakness is caused by missing authorization?
User accesses another BMI record and normal user accesses admin route.
5. Which weakness can expose private data?
Sensitive data exposure, broken object level authorization and SQL injection,
6. Which weakness can affect the browser?
XSS
7. Which weakness can affect the database?
SQL injection, negative weight accepted and mass assignment,

8. Which weakness can be prevented by JWT?

Broken object and function level authorization

9. Which weakness cannot be solved by JWT alone?

Negative weight accepted, SQL injection, XSS, sensitive data exposure, mass assignment and authorization issues.

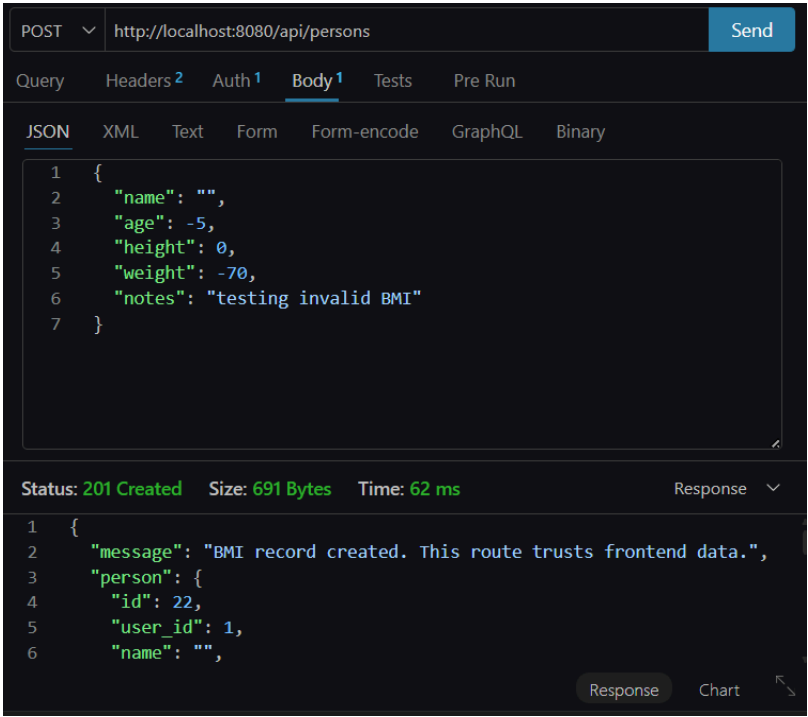
10. Which fixes must be implemented in the backend?

Input validation, authorization checks, prepared statements, JWT verification, mass assignment protection and sensitive data filtering.

Phase 3: Solution Planning

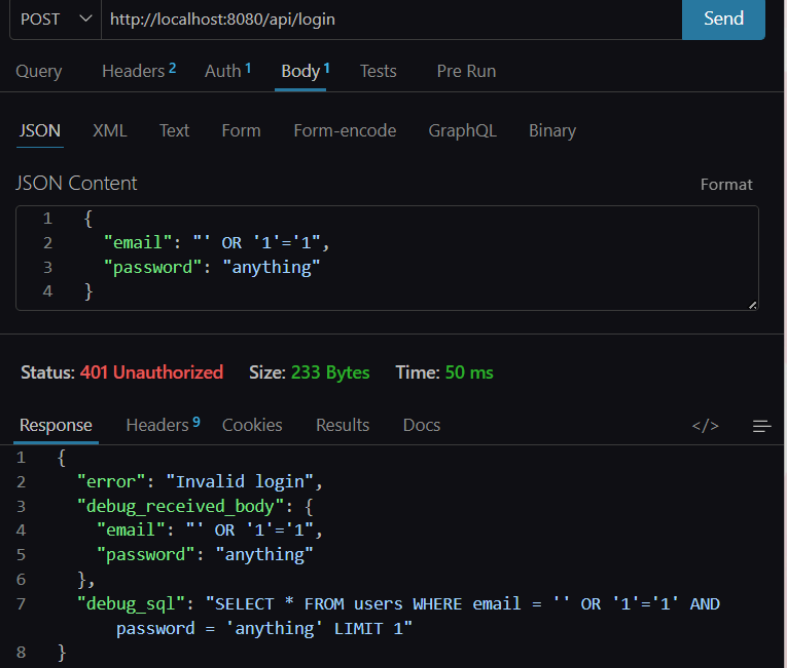
Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI accepted	Backend does not validate	Backend	Validate name, age, height, weight	Submit invalid data again
User accesses other record	No ownership check	Backend API	Compare JWT user_id with record user_id	Access other user's record
User accesses admin route	No role check	Backend API	Check role from verified JWT	Use user token on admin route
password_hash returned	API returns all fields	Backend API	Select only safe fields	Check JSON response
XSS executes	Unsafe rendering	Frontend	Use {{ }} or textContent	Submit XSS payload again
SQL Injection possible	String SQL query	Backend	Use prepared statement	Try injection input again
user_id can be changed	Blind update	Backend API	Allow only safe fields	Send user_id in PUT request

Phase 4: Implementation and Retesting

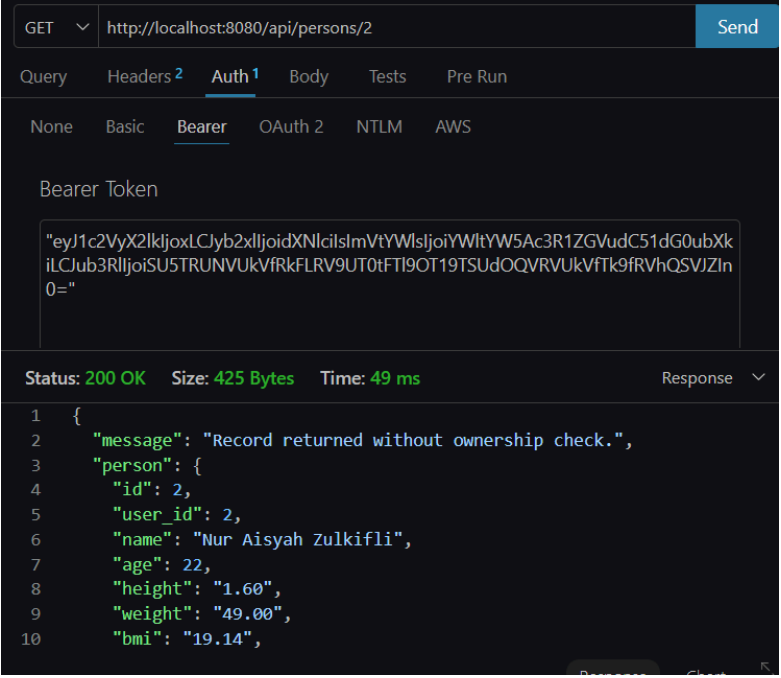
Test Case	Before Fix	After Fix	Evidence
Add BMI with negative weight	Accepted	400 Bad Request	Before fix:  The screenshot shows a REST client interface. At the top, the method is POST and the URL is http://localhost:8080/api/persons. The 'Body' tab is selected, showing a JSON payload: { "name": "", "age": -5, "height": 0, "weight": -70, "notes": "testing invalid BMI" }. Below the body, the status is 201 Created, size is 691 Bytes, and time is 62 ms. The response body is: { "message": "BMI record created. This route trusts frontend data.", "person": { "id": 22, "user_id": 1, "name": "" } }.

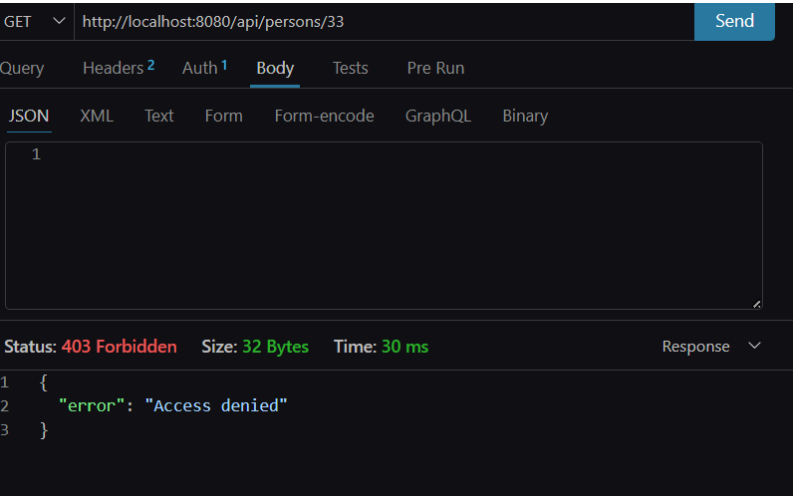
Test Case	Before Fix	After Fix	Evidence
			After fix:  The screenshot shows a REST client interface. At the top, the method is POST and the URL is http://localhost:8080/api/persons. Below the URL bar, there are tabs for Query, Headers, Auth, Body, Tests, and Pre Run. The Body tab is selected, and the content is in JSON format. The JSON body is: { "name": "Negative Weight Test", "age": 22, "height": 1.70, "weight": -70, "notes": "Testing negative weight" }. Below the body, the status is 400 Bad Request, size is 54 Bytes, and time is 54 ms. The response is shown as: { "error": "Weight must be between 2 and 300 kg" }. The backend rejected negative weight and returned 400 Bad Request. This proves backend BMI validation is working.
Submit empty name	Accepted	400 Bad Request	Before fix:

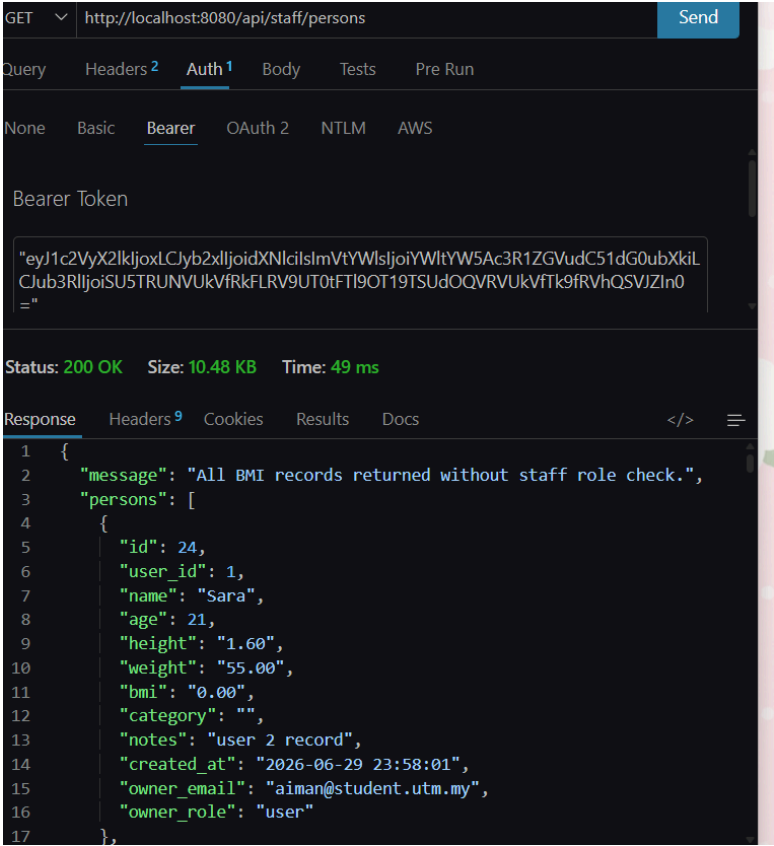
Test Case	Before Fix	After Fix	Evidence
			POST ⌵ http://localhost:8080/api/persons Send Query Headers ² Auth ¹ <u>Body ¹</u> Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary <pre>1 { 2 "name": "", 3 "age": 22, 4 "height": 1.7, 5 "weight": 65, 6 "notes": "empty name test" 7 }</pre> Status: 201 Created Size: 680 Bytes Time: 84 ms Response ⌵ <pre>1 { 2 "message": "BMI record created. This route trusts frontend data.", 3 "person": { 4 "id": 23, 5 "user_id": 1, 6 "name": "", 7 "age": 22, 8 "height": "1.70", 9 "weight": "65.00", 10 "bmi": "0.00", </pre>Response Chart ↗ After fix: POST ⌵ http://localhost:8080/api/persons Send Query Headers ² Auth ¹ <u>Body ¹</u> Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary <pre>1 { 2 "name": "", 3 "age": 22, 4 "height": 1.70, 5 "weight": 65, 6 "notes": "Testing empty name" 7 }</pre> Status: 400 Bad Request Size: 35 Bytes Time: 37 ms Response ⌵ <pre>1 { 2 "error": "Name is required" 3 }</pre>

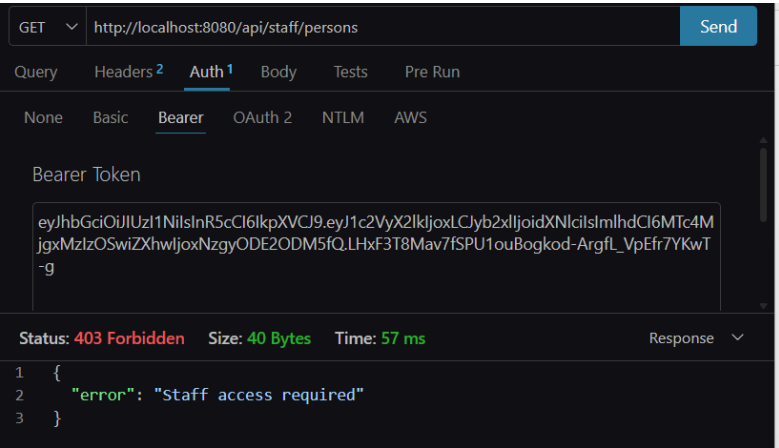
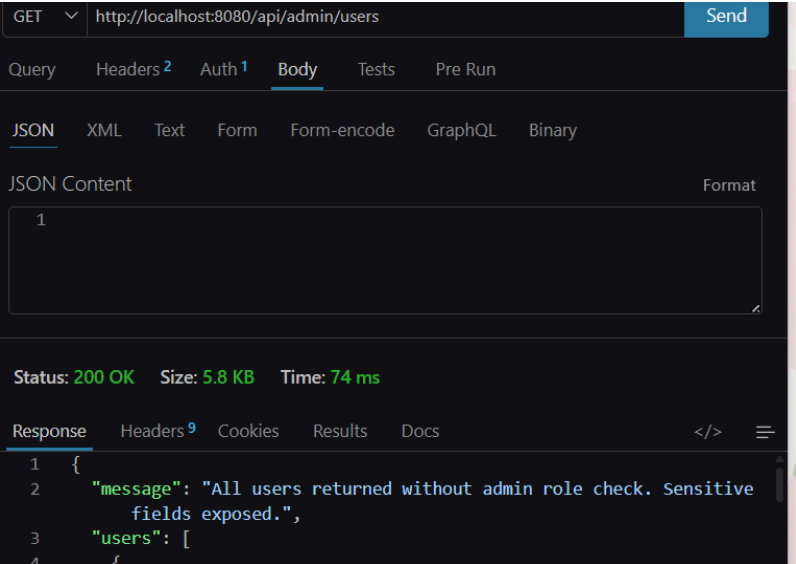
Test Case	Before Fix	After Fix	Evidence
			The backend rejected the request because the name field was empty. This proves required field validation is enforced at the backend.
SQL Injection login input	May affect query	Treated as data	Before fix:  After fix:

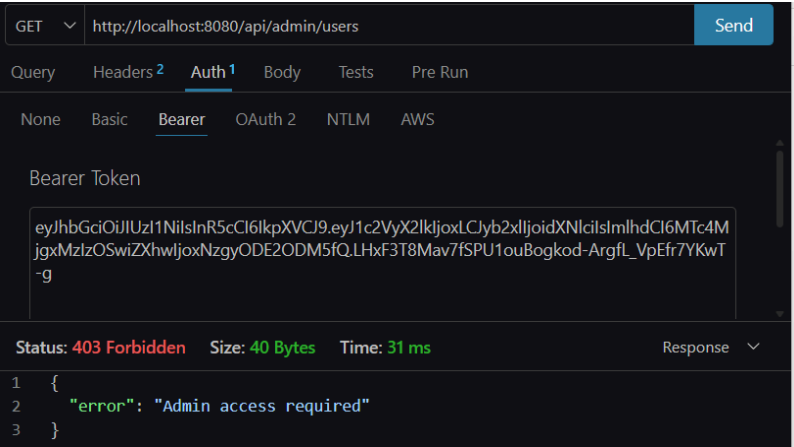
Test Case	Before Fix	After Fix	Evidence
			POST http://localhost:8080/api/login QueryHeaders²Auth¹Body¹TestsPre Run JSONXMLTextFormForm-encodeGraphQLBina <pre>1 { 2 "email": "" OR '1'='1", 3 "password": "anything" 4 }</pre> Status: 401 UnauthorizedSize: 44 BytesTime: 33 ms <pre>1 { 2 "error": "Invalid email or password" 3 }</pre> The SQL injection input was treated as normal text and did not bypass login. This proves the login query uses prepared statements.

Test Case	Before Fix	After Fix	Evidence
Access another user's BMI record	Data returned	403 Access denied	Before fix:  GET http://localhost:8080/api/persons/2 Auth 1 Bearer Token <pre>"eyJ1c2VyX2lkjoxLCjyb2xlbjoidXNlcilslmVtYWwsljoiYWltYW5Ac3R1ZGVudC51dG0ubXkiLCJub3RlIjoSU5TRUNVUkVfRkFLRV9UT0tFTl9OT19TSUdOQVRVUkVfTk9fRVhQSVJZIn0="</pre> Status: 200 OK Size: 425 Bytes Time: 49 ms <pre>{ "message": "Record returned without ownership check.", "person": { "id": 2, "user_id": 2, "name": "Nur Aisyah Zulkifli", "age": 22, "height": "1.60", "weight": "49.00", "bmi": "19.14", } }</pre>

Test Case	Before Fix	After Fix	Evidence
			After fix:  A normal user could not access another user's BMI record. The backend compared the JWT user_id with the record owner_id and returned 403 Access Denied.
Access staff route as user	Allowed	403 Staff access required	Before fix:

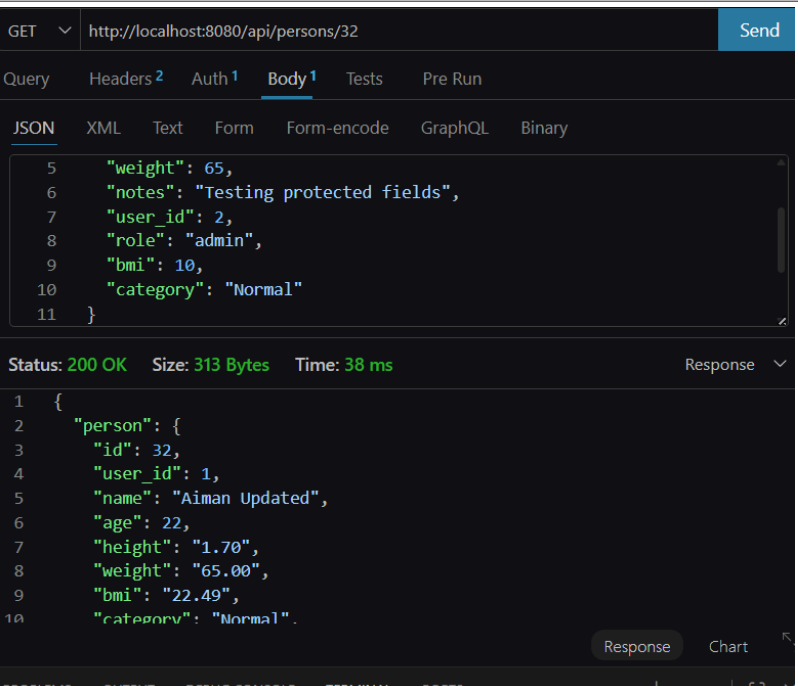
Test Case	Before Fix	After Fix	Evidence
			 GET http://localhost:8080/api/staff/persons Query Headers Auth 1 Body Tests Pre Run None Basic Bearer OAuth 2 NTLM AWS Bearer Token <pre>"eyJ1c2VyX2lkjloxLCJyb2xljoidXNlciIsImVtYWlsbjoiYWltYW5Ac3R1ZGVudC51dG0ubXkilCJub3RlljoiSU5TRUNVUkVfrkFLRV9UT0tFTl9OT19TSUdOCQVRVUkVftk9fRVhQSjVZln0="</pre> Status: 200 OK Size: 10.48 KB Time: 49 ms Response Headers 9 Cookies Results Docs <pre>1 { 2 "message": "All BMI records returned without staff role check.", 3 "persons": [4 { 5 "id": 24, 6 "user_id": 1, 7 "name": "Sara", 8 "age": 21, 9 "height": "1.60", 10 "weight": "55.00", 11 "bmi": "0.00", 12 "category": "", 13 "notes": "user 2 record", 14 "created_at": "2026-06-29 23:58:01", 15 "owner_email": "aiman@student.utm.my", 16 "owner_role": "user" 17 }, 18] 19 }</pre>

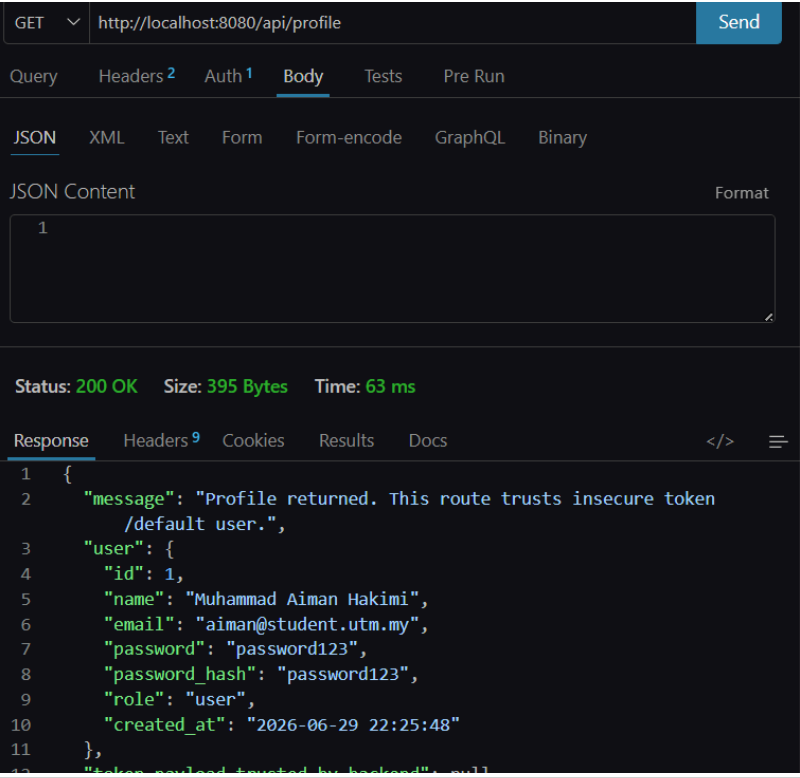
Test Case	Before Fix	After Fix	Evidence
			After fix:  The normal user was blocked from accessing the staff route. This proves role-based access control was applied to staff functions.
Access admin route as user	Allowed	403 Admin access required	Before fix: 

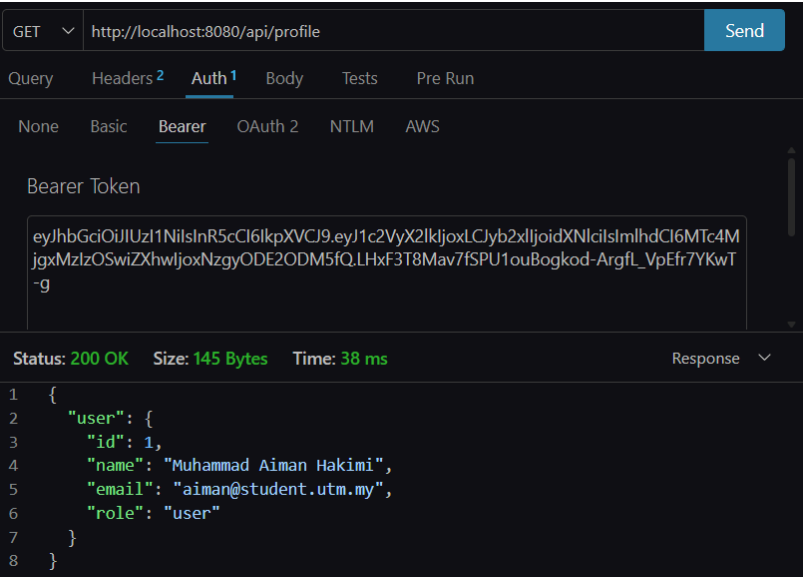
Test Case	Before Fix	After Fix	Evidence
			After fix:  The normal user was blocked from accessing the admin route. This proves admin-only routes are protected by role checking.
Update user_id or role	Accepted	Rejected or ignored	Before fix: 

Test Case	Before Fix	After Fix	Evidence
			GET ⌵ http://localhost:8080/api/persons/1 Send Query Headers ² Auth ¹ Body Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary JSON Content Format 1 Status: 200 OK Size: 432 Bytes Time: 59 ms Response Headers ⁹ Cookies Results Docs 1 { 2 "message": "Record returned without ownership check.", 3 "person": { 4 "id": 1, 5 "user_id": 2, 6 "user_name": "John Doe", 7 "email": "john.doe@example.com", 8 "password": "12345678", 9 "phone": "1234567890", 10 "address": "123 Main St, New York, NY 10001", 11 "created_at": "2023-01-01T00:00:00Z", 12 "updated_at": "2023-01-01T00:00:00Z", 13 }, 14 "status": "success", 15 "timestamp": "2023-01-01T00:00:00Z", 16 } After fix:

Test Case	Before Fix	After Fix	Evidence
			PUT  http://localhost:8080/api/persons/32 Send Query Headers ² Auth ¹ Body ¹ Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary 2 3 4 5 6 7 8 "name": "Aiman Updated", "age": 22, "height": 1.70, "weight": 65, "notes": "Testing protected fields", "user_id": 2, "role": "admin", Status: 200 OK Size: 363 Bytes Time: 97 ms Response  4 5 6 7 8 9 10 11 12 13 "id": 32, "user_id": 1, "name": "Aiman Updated", "age": 22, "height": "1.70", "weight": "65.00", "bmi": "22.49", "category": "Normal", "notes": "Testing protected fields", "created at": "2026-06-30 17:55:49" Response Chart 

Test Case	Before Fix	After Fix	Evidence
			 <pre>GET http://localhost:8080/api/persons/32 Send Query Headers 2 Auth 1 Body 1 Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary 5 "weight": 65, 6 "notes": "Testing protected fields", 7 "user_id": 2, 8 "role": "admin", 9 "bmi": 10, 10 "category": "Normal" 11 } Status: 200 OK Size: 313 Bytes Time: 38 ms Response 1 { 2 "person": { 3 "id": 32, 4 "user_id": 1, 5 "name": "Aiman Updated", 6 "age": 22, 7 "height": "1.70", 8 "weight": "65.00", 9 "bmi": "22.49", 10 "category": "Normal". 11 }</pre> The backend ignored protected fields such as user_id, role, bmi, and category. Only safe fields were updated, and BMI/category were recalculated by the backend.

Test Case	Before Fix	After Fix	Evidence
API returns password_hash	Visible	Removed	Before fix:  The screenshot shows a REST client interface. At the top, the method is GET and the URL is http://localhost:8080/api/profile. Below the URL bar are tabs for Query, Headers, Auth, Body, Tests, and Pre Run. The Body tab is selected, and within it, the JSON format is chosen. The JSON content area shows a single object. Below this, the status is 200 OK, size is 395 Bytes, and time is 63 ms. At the bottom, the Response tab is selected, displaying the JSON response: <pre>1 { 2 "message": "Profile returned. This route trusts insecure token /default user.", 3 "user": { 4 "id": 1, 5 "name": "Muhammad Aiman Hakimi", 6 "email": "aiman@student.utm.my", 7 "password": "password123", 8 "password_hash": "password123", 9 "role": "user", 10 "created_at": "2026-06-29 22:25:48" 11 }, 12 "token_payload_trusted_by_backend": null }</pre>

Test Case	Before Fix	After Fix	Evidence
			After fix:  The API response no longer returned password_hash. Only safe fields such as id, name, email, and role were returned.
XSS payload in notes	Alert executes	Displayed as text	Before fix:

Test Case	Before Fix	After Fix	Evidence
			POST http://localhost:8080/api/persons Send Query Headers 2 Auth 1 Body 1 Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary JSON Content Format 1 { 2 "name": "XSS Test", 3 "age": 22, 4 "height": 1.70, 5 "weight": 65, 6 "notes": "" 7 } Status: 201 Created Size: 743 Bytes Time: 70 ms Response Headers 9 Cookies Results Docs </> ≡ 1 { 2 "message": "BMI record created. This route trusts frontend data.", 3 "person": { 4 "id": 25, 5 "user_id": 1, 6 "name": "XSS Test", 7 "age": 22, After fix:

Test Case	Before Fix	After Fix	Evidence
			POST http://localhost:8080/api/persons Send Query Headers 2 Auth 1 Body 1 Tests Pre Run JSON XML Text Form Form-encode GraphQL Binary <pre>1 { 2 "name": "XSS Test", 3 "age": 22, 4 "height": 1.70, 5 "weight": 65, 6 "notes": "" 7 }</pre> Status: 201 Created Size: 362 Bytes Time: 36 ms Response <pre>1 { 2 "message": "BMI record created successfully", 3 "person": { 4 "id": 35, 5 "user_id": 1, 6 "name": "XSS Test", 7 "age": 22, 8 "height": "1.70", 9 "weight": "65.00", 10 "bmi": "22.49" 11 }</pre> The XSS payload was displayed as normal text and did not execute. This proves the frontend no longer renders user-generated notes as HTML.

API Endpoints Tested

No.	Method	API Endpoint	Purpose of Testing	Expected Secure Result
1	POST	/api/register	Register a new user account	User is created with hashed password, and sensitive fields are not exposed
2	POST	/api/login	Login and obtain JWT token	Valid user receives JWT token; invalid or SQL injection input is rejected
3	GET	/api/profile	Check own user profile	Only safe fields such as id, name, email, and role are returned
4	GET	/api/persons	View own BMI records	Only the logged-in user's BMI records are returned
5	POST	/api/persons	Add a new BMI record	Backend validates input and calculates BMI/category automatically
6	GET	/api/persons/{id}	View one BMI record	Only owner, staff, or admin can view the record
7	PUT	/api/persons/{id}	Update BMI record	Only safe fields are updated; protected fields like user_id, role, bmi, and category are rejected or ignored
8	DELETE	/api/persons/{id}	Delete own BMI record	Only the owner or admin can delete the record
9	GET	/api/staff/persons	Test staff route access	Only staff or admin can view all BMI records
10	GET	/api/staff/persons/{id}	Test staff access to a specific BMI record	Only staff or admin can view any user's BMI record

11	GET	/api/admin/users	Test admin user list access	Only admin can view the user list, and password_hash is not returned
12	PUT	/api/admin/users/{id}/role	Test role update function	Only admin can change a user's role
13	DELETE	/api/admin/persons/{id}	Test admin delete function	Only admin can delete any BMI record

Github Commit Summary

Commit No.	Commit Message	Expected Progress
Commit 1	Initial project setup and security investigation	Vue frontend, backend, and database running. Initial weaknesses tested and recorded.
Commit 2	Add validation password hashing and prepared statements	Fixed backend validation, password hashing, and unsafe SQL queries.
Commit 3	Add JWT authentication and protected routes	Added JWT login, token verification, and protected API routes.
Commit 4	Add RBAC ownership checks and final security fixes	Added role checks, ownership checks, safe responses, error handling, protected field filtering, and XSS fix.
Commit 5	Add database import and final report	Added exported database .sql file and final PDF report with testing evidence and screenshots.

Reflection

1. Which weakness was easiest to find? Why?

The easiest weakness to find was the invalid BMI data validation issue. This was easy because I only needed to send invalid values such as negative weight, empty name, zero height, or invalid age using Thunder Client. The backend accepted the invalid data before the fix, which clearly showed that backend validation was missing.

2. Which weakness was most dangerous? Why?

The most dangerous weakness was broken access control, especially when a normal user could access another user's BMI record or access staff/admin routes. This is dangerous because it can expose private user health data and allow unauthorized users to perform actions that should only be available to staff or admin.

3. Why is frontend validation not enough?

Frontend validation is not enough because users can bypass the frontend by sending requests directly to the API using tools such as Thunder Client or Postman. Even if the form blocks invalid input, attackers can still manually send invalid or manipulated data to the backend. Therefore, validation must always be enforced at the backend.

4. Why should BMI calculation be performed at the backend?

BMI calculation should be performed at the backend because BMI and category are important business logic. If the frontend calculates and sends BMI, users can modify the request and submit fake BMI values or categories. By calculating BMI at the backend, the system ensures that the BMI result is accurate and cannot be manipulated by the user.

5. What is the difference between authentication and authorization?

Authentication and authorization are different. Authentication checks who the user is, usually through login and JWT token verification. Authorization checks what the user is allowed to access or perform. For example, a user may be authenticated after login, but they should not be authorized to access admin routes.

6. What is the difference between RBAC and owner-based access control?

RBAC and owner-based access control also serve different purposes. RBAC controls access based on user roles such as user, staff, and admin. Owner-based access control checks whether the requested data belongs to the current logged-in user. For example, a normal user can manage only their own BMI records, while staff or admin may have wider access depending on the system rules.

7. Why is JWT alone not enough to protect all data?

JWT alone is not enough to protect all data because JWT only proves the identity and role of the logged-in user after verification. The backend must still check permissions for every protected route. For example, even if a user has a valid JWT, the backend must still check whether the user owns the BMI record or has the correct role to access staff/admin functions.

8. Why should API responses exclude password_hash?

API responses should exclude password_hash because the frontend does not need it. Even though password_hash is not the plain password, it is still sensitive information. If exposed, it may increase security risk if attackers obtain the response. The API should only return necessary fields such as id, name, email, and role.

9. Why is Vue route guard not enough to protect backend routes?

Vue route guard is not enough to protect backend routes because it only controls navigation in the frontend. A user can bypass the frontend by calling the backend API directly using Thunder Client or Postman. Therefore, the backend must still verify JWT, user role, and record ownership before returning data or performing actions.

10. How did you prove that your security fix works?

The security fixes were proven by retesting the same API endpoints after implementation. Thunder Client was used to test invalid BMI data, SQL injection input, protected routes, owner-based access control, sensitive data exposure, and protected field updates. Browser testing was used for the XSS payload. The after-fix results showed secure behavior such as 400 Bad Request, 401 Unauthorized, 403 Access Denied, password_hash removed from response, protected fields ignored, and XSS payload displayed as plain text.